AGENDA 8

Estrutura de Repetição

SEAD- Superintendência de Ensino e Pesquisa nas modalidades EaD e Aberta

GOVERNO DO ESTADO DE SÃO PAULO

EIXO TECNOLÓGICO DE INFORMAÇÃO E COMUNICAÇÃO

CURSO TÉCNICO EM DESENVOLVIMENTO DE SISTEMAS

DESENVOLVIMENTO DE SISTEMAS I

Expediente

Autores:

PAULO EDUARDO CARDOSO ANDRADE

TATIANE TOLENTINO DE ASSIS

Revisão Técnica:
Kelly Cristiane de Oliveira Dal Pozzo

São Paulo – SP, 2025

MOMENTO REFLEXÃO

Pense por um instante no seu dia a dia:

Quantas tarefas você repete todos os dias, quase no automático?

Despertar, arrumar a cama, verificar mensagens, organizar materiais, revisar algo, enviar relatórios, preencher planilhas, digitar as mesmas informações várias vezes.

Algumas dessas tarefas são simples, mas quando se repetem demais, tornam-se cansativas, demoradas e até frustrantes.

Agora imagine se você tivesse que repetir a mesma ação centenas de vezes.

Ou milhares.

Ou milhões.

Só de imaginar, já dá uma sensação de desgaste, não dá?

É nesse ponto que nasce a reflexão:

E se existisse uma forma de deixar o computador repetir aquilo que nós não queremos (ou não conseguimos) fazer tantas vezes?

Se você pudesse apertar um botão e deixar:

- um cálculo ser repetido automaticamente,
- uma busca se repetir até encontrar o resultado,
- um processo ser executado quantas vezes fosse necessário…

Como isso mudaria o seu trabalho?

Ou seus projetos?

Ou até sua rotina?

POR QUE APRENDER?

Agora pense no computador.

Ele é uma máquina feita para executar tarefas rapidamente, mas, sem instruções adequadas, ele também dependeria de repetições manuais.

E aí surge a pergunta:

O que aconteceria se nossos programas precisassem ser escritos linha por linha, manualmente, sem repetir nada automaticamente?

Seria inviável!

Pense:

- Um cadastro de 100 pessoas exigiria 100 blocos de código iguais.
- Um gráfico exigiria um trecho repetido para cada barra.

- Um jogo exigiria descrever manualmente cada movimento, quadro por quadro.
- Qualquer análise de dados dependeria de você escrever a mesma instrução milhares de vezes.

Percebe como isso vai contra a natureza da tecnologia?

Um dos motivos dos computadores terem sido criado, foi para justamente para automatizar o que é repetitivo, deixando para nós apenas a lógica e a decisão.

Pense sobre isso:

- Onde, na sua rotina, você depende de repetição para funcionar?
- Que tarefas seriam impossíveis sem automatizar ciclos?
- Como a tecnologia — aplicativos, sites, sensores, sistemas — usa repetições para tomar decisões rápidas e consistentes?

Estruturas de repetição existem porque não precisamos (e não devemos) fazer manualmente aquilo que um computador pode repetir de forma eficiente.

Assista ao vídeo:

Disponível em: https://www.youtube.com/watch?v=LZgMXBVl7uk
Acessado em 02/04/2026.

PARA COMEÇAR O ASSUNTO

Depois de refletir sobre como repetir tarefas manualmente pode ser cansativo e até inviável em diversas situações, vale a pena imaginar como esse mesmo desafio aparece no contexto de um sistema computacional.

Pense, por exemplo, em um aplicativo de mensagens que precisa verificar constantemente se há novas notificações ou em um sistema de controle de estoque que precisa atualizar, diversas vezes ao dia, a quantidade de produtos disponíveis. Nessas situações, repetir ações não é apenas útil, mas essencial para o funcionamento adequado do sistema.

Diferente de nós, que conseguimos repetir tarefas quase automaticamente no cotidiano, o computador depende totalmente das instruções que recebe. Ele não se cansa, não perde o foco e não esquece etapas, mas também não possui iniciativa própria. Isso significa que toda repetição precisa ser cuidadosamente planejada e descrita por quem está desenvolvendo o programa. Em outras palavras, é necessário definir claramente quando uma ação deve começar, o que exatamente deve ser repetido e em que momento essa repetição deve parar.

Ao desenvolver um sistema, você perceberá que essas decisões fazem parte do raciocínio lógico necessário para resolver problemas computacionais. Imagine, por exemplo, a criação de um programa que exiba uma mensagem várias vezes na tela. Para isso, você precisa indicar ao computador quantas vezes essa ação deve ocorrer ou sob qual condição ela deve continuar sendo executada. É nesse contexto que surgem estruturas específicas da programação que permitem automatizar repetições de forma organizada e eficiente.

Em linguagens como Python, esse tipo de situação pode ser representado por comandos que orientam o computador a executar uma mesma instrução diversas vezes, seguindo critérios previamente definidos. Assim, o programador deixa de pensar apenas na ação isolada e passa a estruturar a lógica de repetição, considerando tanto a execução quanto o seu encerramento.

Nesta agenda, você terá a oportunidade de compreender como essas estruturas aparecem na prática e como desenvolver um raciocínio capaz de identificar o que deve ser repetido em um programa e quais condições determinam o início e o fim dessas repetições. Esse é um passo importante para evoluir na construção de algoritmos mais eficientes e alinhados às demandas do mundo do trabalho.

MERGULHANDO NO TEMA

Em programação, é bastante comum encontrar situações em que um mesmo conjunto de instruções precisa ser executado repetidas vezes. Esse tipo de necessidade surge em diversos contextos, como ao percorrer listas de dados, realizar análises automatizadas, gerar relatórios, atualizar informações em sistemas ou executar cálculos contínuos ao longo do tempo. Em todos esses casos, a repetição deixa de ser apenas uma

conveniência e passa a ser um elemento essencial para a construção de soluções eficientes.

Ao contrário dos seres humanos, que podem se cansar ou perder a concentração ao repetir uma mesma tarefa diversas vezes, o computador é capaz de executar instruções repetitivas com extrema precisão e consistência. No entanto, essa capacidade não é automática. Para que a repetição ocorra de maneira correta, é necessário que o programador descreva com clareza todas as condições envolvidas nesse processo. Isso inclui definir o momento em que a repetição deve começar, quais ações devem ser executadas a cada ciclo e em que situação essa repetição deve ser encerrada.

Esse processo de organização lógica exige que você desenvolva um olhar atento para a estrutura do problema, identificando padrões de repetição e transformando esses padrões em instruções compreensíveis para o computador. Em outras palavras, programar não é apenas escrever comandos, mas estruturar o pensamento de forma que o computador consiga executar tarefas de maneira previsível e controlada.

É nesse contexto que surgem as chamadas estruturas de repetição, também conhecidas como loops. Essas estruturas permitem que um bloco de código seja executado múltiplas vezes, de acordo com critérios previamente estabelecidos. Ao dominar esse conceito, você passa a ter maior controle sobre o fluxo de execução de um programa, podendo criar soluções mais dinâmicas, eficientes e adaptáveis a diferentes situações.

Entre os diversos tipos de estruturas de repetição existentes nas linguagens de programação, destacam-se aquelas que permitem controlar a repetição com base em uma quantidade definida de execuções ou em uma condição lógica que determina sua continuidade. Compreender essas variações é fundamental para escolher a abordagem mais adequada em cada problema e construir algoritmos mais robustos e organizados.

Principais estruturas de repetição em Python

Ao aprofundar o estudo das estruturas de repetição em Python, é importante compreender que diferentes situações exigem diferentes formas de controle do fluxo de execução.

Estrutura For (Para)

A estrutura de repetição conhecida como for é amplamente utilizada quando se sabe, antecipadamente, quantas vezes uma determinada ação deve ser executada. Esse tipo de estrutura é especialmente útil em situações que envolvem a manipulação de listas, sequências numéricas ou coleções de dados. Por exemplo, ao desenvolver um sistema que precisa processar uma lista de usuários cadastrados, o comando FOR permite percorrer cada elemento dessa lista de forma organizada, executando uma mesma ação para todos os itens.

Em Python, esse comportamento é frequentemente associado ao uso da função range(), que define um intervalo de valores a ser percorrido. Quando utilizamos uma construção que percorre um intervalo de zero até um determinado número, estamos, na prática, instruindo o computador a repetir uma ação um número exato de vezes. Durante esse processo, uma variável de controle assume valores diferentes a cada ciclo, permitindo que o programa acompanhe o progresso da repetição e, se necessário, utilize essa informação em cálculos ou decisões internas. Veja o exemplo em Python:

python

```python
for i in range(10):
    print("Executando o comando número:", i + 1)
```

Como interpretar:

- range(10) → gera 10 repetições: 0 até 9.
- i → controla cada etapa do ciclo.
- print(...) → é a ação repetida.

Esse exemplo mostra exatamente o caso clássico de repetição controlada por uma quantidade definida de vezes.

Estrutura While (Enquanto)

Existem situações em que não é possível determinar previamente quantas vezes uma repetição deve ocorrer. Nesses casos, utiliza-se a estrutura while, que executa um bloco de código enquanto uma determinada condição for verdadeira. Esse tipo de abordagem é bastante comum em sistemas que dependem da interação com o usuário ou de eventos externos, como validações de entrada de dados, menus interativos ou processos que devem continuar até que um critério específico seja atendido.

Imagine, por exemplo, um programa que solicita ao usuário a digitação de um valor específico para encerrar sua execução. Nesse cenário, o sistema não sabe quantas tentativas serão necessárias até que a condição seja satisfeita. Assim, a repetição continua acontecendo enquanto a condição permanecer verdadeira, sendo interrompida apenas no momento em que o critério de parada é alcançado. Esse comportamento exige atenção especial do programador, pois uma condição mal definida pode levar a repetições infinitas, comprometendo o funcionamento do sistema.

Veja o exemplo em Python:

python

```python
valor = int(input("Digite 5 para continuar: "))

while valor != 5:
    print("Valor incorreto, tente novamente.")
    valor = int(input("Digite 5 para continuar: "))

print("Obrigado! Você digitou 5.")
```

No exemplo apresentado o usuário deve digitar 5 para sair do loop.

Como interpretar:

- O loop começa sem saber quantas repetições serão necessárias.
- Ele só termina quando a condição deixa de ser verdadeira.
- Esse tipo de repetição é ideal para validações, menus e sistemas que dependem da entrada do usuário.

Ao comparar as duas estruturas, percebe-se que a escolha entre for e while está diretamente relacionada à forma como o problema é estruturado. Enquanto o for é mais indicado para repetições com quantidade definida, o while se destaca em situações em que a repetição depende de condições dinâmicas. Desenvolver a capacidade de identificar qual estrutura utilizar em cada contexto é um passo fundamental para a construção de algoritmos mais eficientes, organizados e alinhados às necessidades do mundo real.

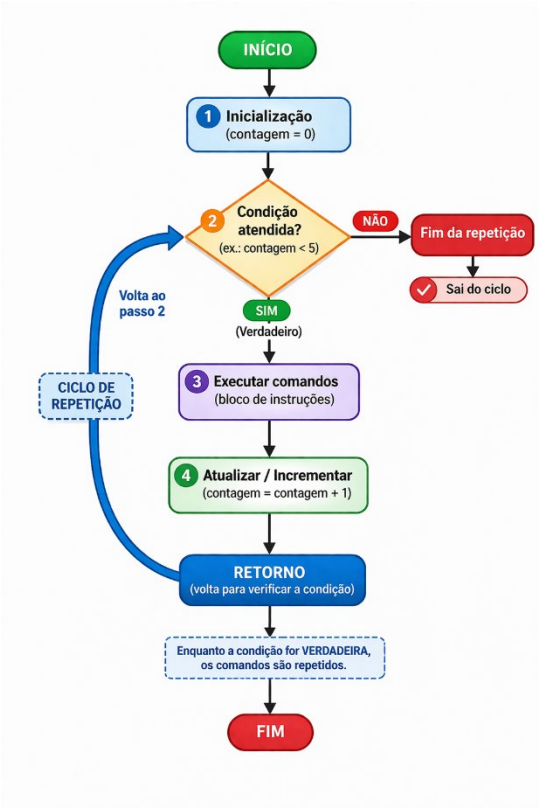
Critério	for	while
Quando usar	Quando sabemos quantas vezes a repetição deve acontecer.	Quando não sabemos quantas vezes vai repetir; depende de uma condição.
Controle do ciclo	Controlado por um contador automático (ex.: range).	Controlado manualmente pelo programador (variáveis, incrementos, condições).
Condição de parada	É definida pelo próprio range ou pela lista percorrida.	Deve ser definida explicitamente na condição do while.
Risco de loop infinito	Baixo, porque o fim geralmente é definido pelo range.	Maior risco se a variável de controle não for atualizada corretamente.
Uso comum	Percorrer listas, repetir tarefas com número fixo de vezes, gerar sequências.	Validação de dados, menus, esperar condições externas, repetir até acertar.
Legibilidade	Código mais curto e previsível quando a repetição é contada.	Mais flexível, mas exige mais cuidado na lógica.
Exemplo mental	“Repita isso 10 vezes.”	“Continue repetindo enquanto isso for verdadeiro.”
Estrutura típica	for item in sequência:	while condição:

Dica:
Use for → quando o número de repetições é conhecido.
Use while → quando a repetição depende de uma condição que pode mudar ao longo do processo.

Entendendo os ciclos da estrutura de repetição

Toda estrutura de repetição segue um ciclo bem definido, que determina como o computador executa uma ação várias vezes. Esse ciclo começa avaliando uma condição ou iniciando um contador. Em seguida, o bloco de instruções é executado, e então ocorre uma atualização, seja aumentando um número, percorrendo uma lista ou verificando novamente uma situação. Depois disso, o ciclo retorna ao início para decidir se continua ou se deve parar.

Esse processo se repete de forma organizada e previsível:



É assim que o computador mantém o controle da repetição, garantindo que tudo aconteça na ordem e na quantidade correta. Entender esse ciclo é essencial para criar programas eficientes, evitar loops infinitos e garantir que cada etapa tenha um propósito claro dentro da lógica construída.

Exemplo Prático

Agora que já entendemos a ideia de repetição, vamos para um exemplo bem clássico: a tabuada. A proposta aqui não é só “ver o resultado funcionando”, mas acompanhar a evolução do código, passo a passo.

Vamos começar com uma versão direta, sem nenhuma estrutura de repetição. Depois, aos poucos, vamos melhorando o código até chegar em uma solução muito mais inteligente, usando for e while.

A ideia é:

1. Primeiro, escrever a tabuada “do jeito difícil”, linha por linha, sem nenhum loop.
2. Depois, deixar o código um pouco mais flexível, permitindo que o usuário escolha o número da tabuada.
3. Em seguida, trocar as várias linhas repetidas por uma estrutura de repetição, que faz o trabalho pesado para nós.
4. Por fim, mostrar como, com uma pequena mudança em um parâmetro, o mesmo código pode gerar 10, 50 ou 100 linhas de tabuada, sem esforço extra.

Acompanhe com calma. A ideia é que você veja claramente o movimento de um código repetitivo e engessado para um código enxuto, automático e escalável.

Vamos lá??

Etapa 1 — Tabuada sem repetição (o jeito “manual”)

Para começar, vamos gerar a tabuada de um número usando apenas linhas diretas de código, sem nenhum tipo de repetição. Veja o código Python:

python

```python
numero = 7

print(numero, "x 1 =", numero * 1)
print(numero, "x 2 =", numero * 2)
print(numero, "x 3 =", numero * 3)
print(numero, "x 4 =", numero * 4)
print(numero, "x 5 =", numero * 5)
print(numero, "x 6 =", numero * 6)
print(numero, "x 7 =", numero * 7)
print(numero, "x 8 =", numero * 8)
print(numero, "x 9 =", numero * 9)
print(numero, "x 10 =", numero * 10)
```

O código funciona, mas claramente repete o mesmo padrão várias vezes.

É como preencher uma planilha copiando e colando a mesma fórmula dez vezes, dá certo, mas não é elegante.

Etapa 2 — Permitindo que o usuário escolha o número

Antes de melhorar o código, vamos deixá-lo um pouco mais flexível: ao invés de sempre usar 7, o usuário decide qual tabuada quer ver.

python

```python
numero = int(input("Digite um número para ver a tabuada: "))

print(numero, "x 1 =", numero * 1)
print(numero, "x 2 =", numero * 2)
print(numero, "x 3 =", numero * 3)
print(numero, "x 4 =", numero * 4)
print(numero, "x 5 =", numero * 5)
print(numero, "x 6 =", numero * 6)
print(numero, "x 7 =", numero * 7)
print(numero, "x 8 =", numero * 8)
print(numero, "x 9 =", numero * 9)
print(numero, "x 10 =", numero * 10)
```

Agora ficou mais interessante!

Mas ainda temos um problema: 10 linhas iguais, mudando só um número no final.

É aqui que começamos a perceber a necessidade da repetição.

Etapa 3 — Usando repetição pela primeira vez

Vamos trocar as dez linhas duplicadas por uma estrutura for.

Ela vai fazer o trabalho repetitivo por nós.

python

```python
numero = int(input("Digite um número para ver a tabuada: "))

for multiplicador in range(1, 11):
    print(numero, "x", multiplicador, "=", numero * multiplicador)
```

Agora sim!

Com apenas duas linhas, resolvemos o que antes exigia dez.

Perceba que:

- range(1, 11) cria os números de 1 a 10.
- multiplicador muda sozinho em cada ciclo.
- O loop executa automaticamente o mesmo padrão.

É como se tivéssemos contratado um assistente que faz o trabalho repetitivo para nós.

Etapa 4 — Usando a repetição com contador (versão com while)

Agora vamos fazer a mesma coisa usando o while, que exige o controle de um contador.

python

```python
numero = int(input("Digite um número para ver a tabuada: "))

contador = 1

while contador <= 10:
    print(numero, "x", contador, "=", numero * contador)
    contador += 1
```

Neste exemplo é possível ver claramente o ciclo acontecendo:

- começamos com contador = 1.
- enquanto a condição for verdadeira, repetimos.
- e no final do ciclo, atualizamos o contador.

Veja o resultado:

Etapa 5 — Mostrando o poder da repetição: alterando apenas um parâmetro

Agora que usamos repetição, podemos facilmente ampliar o programa.

Quer ver a tabuada até 100?

Basta trocar UM número:

Versão utilizando for.

python

```python
numero = int(input("Digite um número para ver a tabuada: "))

for multiplicador in range(1, 101):
    print(numero, "x", multiplicador, "=", numero * multiplicador)
```

Versão utilizando while.

python

```python
numero = int(input("Digite um número para ver a tabuada: "))

multiplicador = 1

while multiplicador <= 100:
    print(numero, "x", multiplicador, "=", numero * multiplicador)
    multiplicador += 1
```

O mesmo código que fazia a tabuada até 10 agora faz até 100, 500 ou 1000.

Sem loops, isso seria impossível sem digitar centenas de linhas manualmente.

Vamos programar!

Imagine que você está desenvolvendo o módulo de lançamento de notas de uma escola. Esse sistema será usado por professores para registrar a média final de cada aluno ao término do bimestre.

Antes de permitir o registro da nota, o sistema precisa garantir regras básicas de funcionamento:

1. A média deve ser um valor válido, entre 0 e 10.
2. Valores negativos ou acima de 10 não podem ser aceitos.
3. O professor só pode prosseguir quando o sistema receber uma média dentro das regras estabelecidas.

Ou seja, se o professor errar a digitação, o sistema deve pedir novamente, até que um valor válido seja informado.

Aqui entra o ciclo da repetição: repetir a solicitação enquanto o valor estiver incorreto.

Etapa 1 — Primeiro lançamento, sem validação.

O sistema aceita qualquer valor que o professor digitar — até mesmo notas impossíveis como -3, 18 ou 200. Isso pode gerar erros graves no boletim.

Precisamos garantir que a nota só seja aceita se estiver dentro do intervalo permitido.

python

```python
senha = input("Digite sua senha: ")

if senha == "etec123":
    print("Acesso liberado!")
else:
    print("Senha incorreta!")
```

Etapa 2 — Identificando o problema.

O sistema precisa validar entradas incorretas automaticamente.

Quais situações devem acionar o alerta?

- média menor que 0
- média maior que 10

Queremos que o sistema repita a pergunta todas as vezes que a média estiver inválida.

Essa é uma situação perfeita para o while.

Etapa 3 — Aplicando validação com WHILE e OR.

Agora vamos aprimorar o sistema para que ele só aceite valores corretos.

python

```python
senha_correta = "etec123"

senha = input("Digite sua senha: ")

while senha != senha_correta:
    print("Senha incorreta! Tente novamente.")
    senha = input("Digite sua senha: ")

print("Acesso liberado! Bem-vindo ao sistema.")
```

Como o sistema funciona:

- O professor informa a média.
- O sistema verifica:
    - Se a média for menor que 0, rejeita.

- Se a média for maior que 10, rejeita.
- Enquanto qualquer um desses problemas ocorrer, o sistema mantém o ciclo:

"Digite novamente…"

- Assim que a média estiver entre 0 e 10, o lançamento é autorizado.

Esse fluxo espelha exatamente o funcionamento de um sistema real: repete, valida, corrige e só então libera.

Usando Operadores Lógicos nas Condições de Repetição

Nas estruturas de repetição (while e, em alguns casos, for), podemos usar operadores lógicos como **and**, **or** e **not** para construir condições mais inteligentes e flexíveis.

Isso é muito útil quando o ciclo depende de **mais de um critério ao mesmo tempo**.

Quando criamos um loop, nem sempre queremos repetir algo baseado em apenas uma condição simples. Em muitos casos, precisamos analisar duas ou mais situações para decidir se a repetição deve continuar.

É aqui que entram os operadores lógicos:

✓ and

Só continua o loop se **todas** as condições forem verdadeiras.

✓ or

Continua o loop se **pelo menos uma** condição for verdadeira.

✓ not

Inverte a condição, permitindo repetir enquanto algo **não** estiver correto.

Por que isso é poderoso?

Usar operadores lógicos permite criar condições muito mais completas, como:

- Validar intervalos de valores
- Repetir até cumprir múltiplas regras
- Verificar diferentes cenários ao mesmo tempo
- Tornar o programa mais seguro e prevenir erros
- Implementar comportamentos reais (como sistemas de login, sensores, validações, menus)

Em outras palavras:

Operadores lógicos tornam a repetição inteligente, porque permitem que o computador avalie várias situações antes de decidir continuar ou parar o ciclo.

Sem o while, o sistema aceitaria valores inválidos ou encerraria o programa de forma abrupta. Com a repetição, ele aprende a "aguardar o valor correto", assim como um sistema profissional.

Antes de prosseguir.....

Agora que você já compreende o funcionamento básico das estruturas de repetição, é importante observar como elas podem ficar ainda mais inteligentes quando combinadas com operadores lógicos.

Esses operadores permitem criar condições mais completas, envolvendo múltiplos critérios ao mesmo tempo algo essencial em sistemas reais.

Na prática, isso significa que um loop pode continuar repetindo enquanto:

- duas condições precisam ser verdadeiras ao mesmo tempo (and),
- pelo menos uma condição estiver incorreta (or),
- uma situação desejada ainda não aconteceu (not).

Para visualizar melhor como essa combinação funciona, veja a seguir uma tabela com exemplos práticos de uso de operadores lógicos dentro de estruturas de repetição.

Ela mostrará situações reais e como cada operador influencia a lógica do loop.

Situação	Condição usada	Operador lógico	Significado
Repetir enquanto a idade for inválida	idade < 0 or idade > 120	**OR**	Repete se **qualquer** uma das condições indicar erro.
Repetir enquanto senha e confirmação não combinarem	senha != confirmacao and tentativas > 0	**AND**	Repete somente se **as duas** condições forem verdadeiras ao mesmo tempo.
Repetir até o usuário aceitar os termos	not aceitou_termos	**NOT**	Repete enquanto o usuário **não** aceitar.
Repetir enquanto temperatura estiver fora do ideal	temp < 18 or temp > 26	**OR**	Repete se a temperatura estiver **abaixo** ou **acima** do intervalo.
Repetir enquanto sistema não estiver pronto	not sistema_pronto	**NOT**	Repete enquanto a condição desejada **não** for atendida.
Repetir enquanto volume alto e microfone ligado	volume > 80 and mic_ligado	**AND**	Repete apenas se **ambos** os fatores forem verdadeiros.

Assista a aula explicativa:

Disponível em: https://www.youtube.com/watch?v=X-YNeq4Bo_0
Acessado em 02/04/2026.